toEqual. The matchers typically take an argument, which is the predetermined expected value—hence expect(1 + 1).toEqual(2).

You can run this from the command line (output slightly edited for space):

```
$ yarn test

webpack: wait until bundle finished:

webpack: Compiled successfully.
04 08 2017 19:53:40.789:INFO [karma]:
        Karma v2.0.0 server started at http://0.0.0.0:9876/
04 08 2017 19:53:40.792:INFO [launcher]:
        Launching browser ChromeHeadless with unlimited concurrency
04 08 2017 19:53:40.797:INFO [launcher]:
        Starting browser ChromeHeadless
04 08 2017 19:53:41.239:INFO [HeadlessChrome 0.0.0 (Mac OS X 10.13.2)]:
        Connected on socket a3N9XHHP4GaY_KiQAAAA with id 80412174

  JavaScript testing
    ✓ works

Finished in 0.004 secs / 0.003 secs @ 19:53:41 GMT-0500 (CDT)

SUMMARY:
✓ 1 test completed
```

The output tells you that Karma set up a server at http://0.0.0.0:9876/, started a headless Chrome instance pointed at that server, and ran the tests. Notice an RSpec documentation–style listing of the tests (in green on a color terminal) and a summary at the end.

If you want to run Karma in a browser or keep the server running, run the command NODE_ENV=test karma start --no-single-run (or change the singleRun option in the Karma configuration file). If Karma is running, you can point any browser you want at the server address and Karma will run tests in that browser. (Output will go to the console.)

TDD in JavaScript

Now let's try to build the task-rearrangement feature in JavaScript using TDD. You'll use a different approach to the JavaScript than you did in the previous chapter, and you'll be completely rewriting your JavaScript Task class. First, you'll build the logic without regard to how the application will get data into the JavaScript objects and without regard to how the application will interact with the DOM. Approaching the feature this way allows you to focus on the code and not the large external dependencies of the server-side data and the DOM.